Props, Props Drilling & Context API

---

1. Props in React

Props (Properties) are used to pass data from a **parent component to a child component**.

👉 Props make components **reusable and dynamic**

```
const App = () => {
  return <User name="Usama" age={22} />
}

const User = ({ name, age }) => {
  return (
    <div>
      <h2>Name: {name}</h2>
      <h3>Age: {age}</h3>
    </div>
  )
}
```

Key Points of Props

- Passed from **Parent → Child**
- Are **read-only (immutable)**
- Use **destructuring** for clean code

◇ Passing Function as Props

```
const App = () => {
  const greet = () => alert("Hello!")

  return <Button onClick={greet} />
}

const Button = ({ onClick }) => {
  return <button onClick={onClick}>Click Me</button>
}
```

👉 Called **Callback Props**

---

2. Props Drilling

Passing props through multiple components that **don’t use them**, just to reach a nested component.

❌ Example

```
const App = () => {
  const name = "Usama"
  return <Parent name={name} />
}

const Parent = ({ name }) => {
  return <Child name={name} />
}

const Child = ({ name }) => {
  return <GrandChild name={name} />
}

const GrandChild = ({ name }) => {
  return <h1>{name}</h1>
}
```

❗ Problems

- Unnecessary code
- Hard to maintain
- Poor scalability

---

3. Context API (Solution)

👉 Used to **avoid props drilling**
👉 Allows global data sharing

Step-by-Step Implementation

Step 1: Create Context

🗀 UserContext.js

```
import { createContext } from "react"

const UserContext = createContext()

export default UserContext
What is this?
```

👉 Creates a **global data container**

Step 2: Provide Data

🗀 App.js

```
import UserContext from "./UserContext"
import Parent from "./Parent"

const App = () => {
  return (
    <UserContext.Provider value="Usama">
      <Parent />
    </UserContext.Provider>
  )
}

export default App
```

🔍 What is Provider?

👉 **Provider = Data Supplier**

- Wraps components
- Shares data using `value`
- Makes data available to all child components

Step 3: Consume Data

🗀 **GrandChild.js**

```
import { useContext } from "react"
import UserContext from "./UserContext"

const GrandChild = () => {
  const name = useContext(UserContext)

  return <h1>{name}</h1>
}

export default GrandChild
What is this?
```

Result

- No props drilling ❌
- Direct data access ☑

---

4. Alternative: Component Composition

```
const App = () => {
  const name = "Usama"
  return <Parent child={<GrandChild name={name} />} />
}

const Parent = ({ child }) => {
  return <div>{child}</div>
}

const GrandChild = ({ name }) => {
  return <h1>{name}</h1>
}
```

---

5. Comparison

Concept	Explanation
Props	Pass data Parent → Child
Props Drilling	Passing through unnecessary components
Context API	Global data sharing

---

6. When to Use

Situation	Use
Simple data passing	Props
Deep nesting	Context API
Large apps	Redux / Zustand

Summary

👉 **Props**

- Used for data passing
- Read-only
- Parent → Child

👉 **Props Drilling**

- Unnecessary prop passing
- Makes code complex

👉 **Context API**

- Avoids props drilling
- Uses Provider & useContext